웹 개발자 부트캠프 과정

SeSAC x CODINGOn

함수에서의 type 선언

함수 선언과 typescript

- 선언시에 타입설정, 호출할 땐 기존처럼

  1. 매개변수 타입설정

  2. 함수의 return 타입에 따라 함수 전체 타입 설정

  (리턴 타입을 보고 타입을 추론할 수 있으므로 생략 가능)

  이미 알고 있는 타입 외에도 never 과 void 를 함수의 리턴 타입으로 설정 가능

선언하는 방식

- 파라미터와 리턴 타입을 선언하는 기본 형식

```
function sum(a: number, b: number): number {
  return a + b;
}
```

왼쪽의 sum 함수는 number 형 a, b를 매개변수로 받아 합을 return 하는 함수입니다.

- 화살표 함수로 타입을 선언

```
const sum = (a: number, b: number): number => {
  return a + b;
}
```

매개변수 a, b의 type 설정

함수의 return type 설정

- 리턴 생략한 형태로도 선언

```
const sum = (a: number, b: number): number => a + b
```

함수와 매개변수의 개수

JS vs.TS

- JS: 매개변수 선언하고, 호출 시 parameter 전달하지 않아도 오류 x
- TS: 매개변수를 2개 선언했다면, 호출했을 때 반드시 모든 값 전달해줘야 함

```
function sum(a, b, c) {
    return a + b + c;
}
console.log(sum(1, 2));
```

```
function sum(a:number, b:number, c:number) {
    return a + b + c;
}
console.log(sum(1, 2));
```

세 개의 매개변수를 가진 함수 sum 을 만들었지만
함수 호출 시 2개의 parameter 만 전달했기 때문에 오류!

들어오지 않을 변수를 처리하기 위해서는 ? 이용

함수와 매개변수의 개수

- 세번째 매개변수인 c에게 값을 전달하지 않는 경우가 생긴다면 ?를 이용해서 undefined가 될 수도 있음을 정의

```
function print(a: number, b: number, c?: number) {
    console.log(a);
    console.log(b);
    console.log(c);
}
print(1, 2);
```

함수의 리턴 타입이 없을 때 void

- void 란 비어있다는 의미입니다.
- 리턴이 없는 함수는 void로 설정할 수 있습니다.

```
function print(a: number, b: number, c?: number):void {
    console.log(a);
    console.log(b);
    console.log(c);
}
```

끝이 없는 함수 never

- 어떤 조건에서도 함수의 끝에 도달할 수 없을 때 사용

```
function goingOn(): never {
    while (true) {
        console.log("go");
    }
}
```

실습. TypeScript 함수 실습

- **[코딩온] React 실습** 강의 클릭
  - ➡ **[TypeScript 함수 실습]** 커리큘럼 클릭
  - ➡ **[1~2번 문제]** 진행

함수와 generic <T>

Generic

- 클래스, 함수, 인터페이스에서 다양한 타입으로 재사용 가능
- 선언할 때는 타입 파라미터만 명시하고, 생성 시점에 사용할 타입을 결정!
- 예를 들면, 타입을 특정할 수 없는 함수가 있다면?
  - 배열을 매개변수로 받아 배열의 길이를 리턴하는 함수라면

```
function arrLength(arr: (number | string | boolean | object | null)[]): number {
  return arrLength.length;
}
```

```
function arrLength(arr: any[]): number {
  return arrLength.length;
}
```

  - 모든 타입의 데이터 타입이 들어올 수 있으므로 위처럼 사용할 수 있음
  - 하지만 모든 데이터타입을 쓰거나 any를 쓰는 것은 typescript 를 사용하는 이유가 사라짐.

Generic

- 함수를 호출할 때 데이터 타입을 지정할 수 있는 문법 Generic!

- 선언

```
function arrLength2<T>(arr:T[]):number{
    return arr.length
}
```

- 함수 호출

```
arrLength2<string>(["a"]);
arrLength2<number>([1, 2, 3, 4]);
```

  - **함수를 호출할 때** 매개변수로 들어갈 데이터 **타입을 설정**
  - 타입을 함수의 파라미터처럼 사용할 수 있음

실습. TypeScript 제너릭 실습

- **[코딩온] React 실습** 강의 클릭
  - ➡ **[TypeScript 제너릭 실습]** 커리큘럼 클릭
  - ➡ **[1~2번 문제]** 진행